PYTHON PROGRAMMING
CASE STUDY RECORD

10 Complete Case Studies (91–100)

Name: Dontha Kowshik
Branch: CSE-AIML
Section: 1

Includes problem statement, objective, assumptions, algorithm, flowchart, pseudocode, Python source code, sample test cases/output, and conclusion for each program.

Contents

91. Advanced String-Based Registration Record Analyzer

92. Multi-Event Sports Scheduling and Scoring

93. Smart Public Transport Fare Reconciliation

94. Disaster Relief Supply Allocation

95. Integrated Manufacturing Production Planner

96. Multi-Criteria Library Resource Distribution

97. Smart Campus Incident and Maintenance Prioritization

98. University Academic Progression Audit

99. Integrated E-Commerce Order Processing and Reconciliation

100. Comprehensive Smart Campus Management Simulation

91. Advanced String-Based Registration Record Analyzer

Problem Statement

Parse structured registration records, validate field lengths and formats, extract components, detect duplicate IDs, and classify errors.

Objective

Practice strings, lists, loops, conditionals, and basic validation.

Assumptions

• Each record uses ID|NAME|EMAIL|PHONE format.

• ID is 4 digits; name is 2–30 characters; email contains @ and a dot after it; phone is 10 digits.

• A repeated ID is classified as a duplicate. Records are illustrative.

Algorithm

1. Read each record and split it on '|'.

2. Check field count, lengths, and formats.

3. Track IDs already seen to detect duplicates.

4. Print extracted fields and any error classifications.

Flowchart

Start → Read records → Split fields → Validate fields → Check duplicate ID → Display classification → Next record? → End

Pseudocode

```
START
SET seen_ids to empty list
FOR each record
  SPLIT record into fields
  IF field count is not 4, report FIELD_COUNT error
  ELSE validate ID, name, email, and phone
  IF ID is already in seen_ids, report DUPLICATE_ID
  ELSE add ID to seen_ids
  DISPLAY fields and errors
END FOR
STOP
```

Python Source Code

```
records = [
    "1001|Asha Rao|asha@example.com|9876543210",
    "1002|Ravi K|ravi.example.com|12345",
    "1001|Meena Das|meena@example.com|9123456780",
    "10|Jo|jo@mail.com|9876543210"
]

seen_ids = []
for record in records:
    fields = record.split("|")
    errors = []
    if len(fields) != 4:
        errors.append("FIELD_COUNT")
    else:
        reg_id, name, email, phone = fields
        if len(reg_id) != 4 or not reg_id.isdigit():
            errors.append("INVALID_ID")
        if len(name) < 2 or len(name) > 30:
            errors.append("INVALID_NAME_LENGTH")
        if "@" not in email or "." not in email[email.find("@") + 1:]:
            errors.append("INVALID_EMAIL")
        if len(phone) != 10 or not phone.isdigit():
            errors.append("INVALID_PHONE")
        if reg_id in seen_ids:
            errors.append("DUPLICATE_ID")
        else:
            seen_ids.append(reg_id)
        print("ID:", reg_id, "| Name:", name, "| Email:", email)
```

```
    print("Errors:", errors if errors else ["NONE"])
```

Test Cases and Sample Output

```
ID: 1001 | Name: Asha Rao | Email: asha@example.com
Errors: ['NONE']
ID: 1002 | Name: Ravi K | Email: ravi.example.com
Errors: ['INVALID_EMAIL', 'INVALID_PHONE']
ID: 1001 | Name: Meena Das | Email: meena@example.com
Errors: ['DUPLICATE_ID']
ID: 10 | Name: Jo | Email: jo@mail.com
Errors: ['INVALID_ID']
```

Conclusion

This program demonstrates practice strings, lists, loops, conditionals, and basic validation. The sample data can be changed to test additional cases.

92. Multi-Event Sports Scheduling and Scoring

Problem Statement

Store sports fixtures and team scores, detect teams scheduled for overlapping slots, total points, and rank teams.

Objective

Use tuples, lists, nested lists, loops, and sorting-free ranking logic.

Assumptions

• A fixture is (event, time_slot, team1, team2).

• A win gives 3 points, a draw gives 1 point to each team, and a loss gives 0.

• Two fixtures conflict for a team if that team appears in the same time slot.

Algorithm

1. Store fixtures as tuples and results as tuples.

2. Compare fixture pairs for same-slot team conflicts.

3. Accumulate points for each team.

4. Select the highest remaining score repeatedly to print rankings.

Flowchart

Start → Read fixtures/results → Check slot conflicts → Add points → Select highest score → Print ranking → End

Pseudocode

```
START
CREATE fixture tuples and result tuples
COMPARE every fixture pair for same time and shared team
INITIALIZE points for every team to zero
FOR each result, award 3/1/0 points
REPEATEDLY select highest unranked score
DISPLAY conflicts and ranking
STOP
```

Python Source Code

```
fixtures = [
    ("Football", "09:00", "Falcons", "Tigers"),
    ("Basketball", "09:00", "Tigers", "Eagles"),
    ("Volleyball", "10:00", "Falcons", "Eagles")
]
results = [
    ("Falcons", "Tigers", 2, 1),
    ("Tigers", "Eagles", 3, 3),
    ("Falcons", "Eagles", 0, 1)
]
teams = ["Falcons", "Tigers", "Eagles"]
points = [0, 0, 0]

print("Scheduling conflicts:")
found = False
for i in range(len(fixtures)):
    for j in range(i + 1, len(fixtures)):
        a, time_a, t1, t2 = fixtures[i]
        b, time_b, u1, u2 = fixtures[j]
        if time_a == time_b and (t1 == u1 or t1 == u2 or t2 == u1 or t2 == u2):
            print(t1, "or", t2, "has a conflict:", a, "and", b, "at", time_a)
            found = True
if not found:
    print("None")

for team1, team2, score1, score2 in results:
    i = teams.index(team1)
    j = teams.index(team2)
    if score1 > score2:
        points[i] += 3
    elif score2 > score1:
        points[j] += 3
    else:
        points[i] += 1
        points[j] += 1

print("Final ranking:")
```

```
ranked = []
for count in range(len(teams)):
    best = -1
    for i in range(len(teams)):
        if teams[i] not in ranked and (best == -1 or points[i] > points[best]):
            best = i
    ranked.append(teams[best])
    print(count + 1, ranked[-1], "-", points[best], "points")
```

Test Cases and Sample Output

```
Scheduling conflicts:
Falcons or Tigers has a conflict: Football and Basketball at 09:00
Final ranking:
1 Eagles - 4 points
2 Falcons - 3 points
3 Tigers - 1 points
```

Conclusion

This program demonstrates use tuples, lists, nested lists, loops, and sorting-free ranking logic. The sample data can be changed to test additional cases.

93. Smart Public Transport Fare Reconciliation

Problem Statement

Calculate fares by route and passenger category, compare expected revenue with actual collections, and report discrepancies.

Objective

Apply arithmetic, conditions, nested lists, and reconciliation.

Assumptions

• Fare table gives adult base fares for each route.

• Students pay 50% and senior citizens pay 60% of the adult fare.

• The sample records and collected revenue are hypothetical.

Algorithm

1. Define route fares and passenger trips.

2. Calculate each trip's fare using category rules.

3. Add expected fares and compare with collected revenue.

4. Print each trip and the discrepancy.

Flowchart

Start → Load fares and trips → Calculate category fare → Total expected → Compare collection → Report discrepancy → End

Pseudocode

```
START
SET route fares
SET trip records and collected revenue
FOR each trip
  FIND route fare
  APPLY passenger-category multiplier
  ADD fare to expected total
END FOR
CALCULATE collected minus expected
DISPLAY totals and discrepancy
STOP
```

Python Source Code

```
route_fares = [("R1", 20), ("R2", 35), ("R3", 50)]
trips = [
    ("R1", "adult", 10),
    ("R2", "student", 8),
    ("R3", "senior", 4)
]
collected_revenue = 600
expected = 0

for route, category, passengers in trips:
    base_fare = 0
    for code, fare in route_fares:
        if code == route:
            base_fare = fare
    if category == "adult":
        fare = base_fare
    elif category == "student":
        fare = base_fare * 0.5
    elif category == "senior":
        fare = base_fare * 0.6
    else:
        fare = 0
    amount = fare * passengers
    expected += amount
    print(route, category, "passengers:", passengers, "amount:", amount)

difference = collected_revenue - expected
print("Expected revenue:", expected)
print("Collected revenue:", collected_revenue)
print("Discrepancy:", difference)
if difference == 0:
```

```
    print("Status: Reconciled")
elif difference > 0:
    print("Status: Excess collection")
else:
    print("Status: Short collection")
```

Test Cases and Sample Output

```
R1 adult passengers: 10 amount: 200
R2 student passengers: 8 amount: 140.0
R3 senior passengers: 4 amount: 120.0
Expected revenue: 460.0
Collected revenue: 600
Discrepancy: 140.0
Status: Excess collection
```

Conclusion

This program demonstrates apply arithmetic, conditions, nested lists, and reconciliation. The sample data can be changed to test additional cases.

94. Disaster Relief Supply Allocation

Problem Statement

Compare supply stock with area-wise demand, prioritize urgent shortages, allocate limited stock, and calculate unmet demand.

Objective

Use nested lists, priorities, loops, and arithmetic for allocation.

Assumptions

• Each area record is [area, item, demand, urgency].

• Urgency 1 is highest priority, then 2, then 3.

• Stock is allocated to the same item in priority order.

Algorithm

1. Create area demand records and stock records.

2. Process urgency levels from 1 to 3.

3. Allocate the smaller of available stock and demand.

4. Update stock and calculate unmet demand.

Flowchart

Start → Read stock and demands → Process urgency order → Allocate stock → Update balance → Report unmet demand → End

Pseudocode

```
START
SET stock and area demands
FOR urgency from 1 to 3
  FOR each demand with that urgency
    ALLOCATE minimum(stock, demand)
    REDUCE stock and demand by allocation
    RECORD unmet demand
DISPLAY allocation and remaining stock
STOP
```

Python Source Code

```
stock = [["water", 100], ["food", 60]]
demands = [
    ["North", "water", 50, 1],
    ["South", "water", 40, 2],
    ["East", "water", 30, 3],
    ["North", "food", 35, 1],
    ["South", "food", 40, 2]
]
allocations = []

for priority in [1, 2, 3]:
    for area, item, demand, urgency in demands:
        if urgency == priority:
            available = 0
            for stock_item, quantity in stock:
                if stock_item == item:
                    available = quantity
            allocated = min(available, demand)
            for row in stock:
                if row[0] == item:
                    row[1] -= allocated
            unmet = demand - allocated
            allocations.append([area, item, allocated, unmet, priority])

print("Area | Item | Allocated | Unmet | Priority")
for row in allocations:
    print(row[0], "|", row[1], "|", row[2], "|", row[3], "|", row[4])
print("Remaining stock:", stock)
```

Test Cases and Sample Output

```
Area | Item | Allocated | Unmet | Priority
North | water | 50 | 0 | 1
```

```
North | food | 35 | 0 | 1
South | water | 40 | 0 | 2
South | food | 25 | 15 | 2
East | water | 10 | 20 | 3
Remaining stock: [['water', 0], ['food', 0]]
```

Conclusion

This program demonstrates use nested lists, priorities, loops, and arithmetic for allocation. The sample data can be changed to test additional cases.

95. Integrated Manufacturing Production Planner

Problem Statement

Evaluate raw material availability, production demand, defects, and order feasibility.

Objective

Use lists, tuples, arithmetic, and conditionals to plan production and quality checks.

Assumptions

• Each unit requires 2 kg of material A and 1 unit of material B.

• A production order is feasible only if both materials suffice.

• Defect rate above 5% is a quality violation.

Algorithm

1. Read available materials and requested production.

2. Calculate the maximum production possible from each material.

3. Determine feasible quantity and shortages.

4. Calculate good units and defect rate; report quality status.

Flowchart

Start → Read materials and demand → Calculate material limits → Check feasibility → Calculate defects → Report shortages/quality → End

Pseudocode

```
START
SET material stock, demand, and defects
CALCULATE material-limited quantities
SET feasible quantity to minimum of limits and demand
CALCULATE material shortages
CALCULATE defect rate
IF defect rate > 5%, report quality violation
DISPLAY production plan
STOP
```

Python Source Code

```
material_a_kg = 180
material_b_units = 70
order_demand = 100
defects = 4

limit_a = material_a_kg // 2
limit_b = material_b_units
production = min(order_demand, limit_a, limit_b)
shortage = order_demand - production
used_a = production * 2
used_b = production
defect_rate = defects / production * 100 if production > 0 else 0
good_units = production - defects

print("Order demand:", order_demand)
print("Feasible production:", production)
print("Unmet order quantity:", shortage)
print("Material A remaining (kg):", material_a_kg - used_a)
print("Material B remaining:", material_b_units - used_b)
print("Good units:", good_units)
print("Defect rate:", round(defect_rate, 2), "%")
if defect_rate > 5:
    print("Quality status: VIOLATION")
else:
    print("Quality status: ACCEPTABLE")
```

Test Cases and Sample Output

```
Order demand: 100
Feasible production: 70
Unmet order quantity: 30
Material A remaining (kg): 40
Material B remaining: 0
Good units: 66
```

```
Defect rate: 5.71 %
Quality status: VIOLATION
```

Conclusion

This program demonstrates use lists, tuples, arithmetic, and conditionals to plan production and quality checks. The sample data can be changed to test additional cases.

96. Multi-Criteria Library Resource Distribution

Problem Statement

Allocate available book copies to branches using priority rules and calculate unmet borrowing demand.

Objective

Model branch requirements with nested lists and allocate resources in priority order.

Assumptions

• Each requirement is [branch, title, copies_needed, priority].

• Priority 1 is highest; stock is allocated in that order.

• Only matching titles can satisfy a requirement.

Algorithm

1. Set title stock and branch demands.

2. Process demands by priority.

3. Allocate available copies up to requested copies.

4. Update stock and report unmet demand.

Flowchart

Start → Read title stock and demands → Sort by priority rule → Allocate copies → Update stock → Report unmet demand → End

Pseudocode

```
START
SET stock and branch demand records
FOR priority from 1 to 3
  FOR each demand at this priority
    FIND available copies for title
    ALLOCATE minimum(available, requested)
    UPDATE stock and unmet demand
DISPLAY allocations and stock balance
STOP
```

Python Source Code

```
stock = [["Python Basics", 12], ["AI Fundamentals", 7]]
requests = [
    ["Central", "Python Basics", 6, 1],
    ["East", "Python Basics", 5, 2],
    ["West", "Python Basics", 4, 3],
    ["Central", "AI Fundamentals", 4, 1],
    ["East", "AI Fundamentals", 5, 2]
]
print("Branch | Title | Allocated | Unmet")
for priority in [1, 2, 3]:
    for branch, title, needed, level in requests:
        if level == priority:
            available = 0
            for row in stock:
                if row[0] == title:
                    available = row[1]
            allocated = min(available, needed)
            for row in stock:
                if row[0] == title:
                    row[1] -= allocated
            print(branch, "|", title, "|", allocated, "|", needed - allocated)
print("Remaining stock:", stock)
```

Test Cases and Sample Output

```
Branch | Title | Allocated | Unmet
Central | Python Basics | 6 | 0
Central | AI Fundamentals | 4 | 0
East | Python Basics | 5 | 0
East | AI Fundamentals | 3 | 2
West | Python Basics | 1 | 3
Remaining stock: [['Python Basics', 0], ['AI Fundamentals', 0]]
```

Conclusion

This program demonstrates model branch requirements with nested lists and allocate resources in priority order. The sample data can be changed to test additional cases.

97. Smart Campus Incident and Maintenance Prioritization

Problem Statement

Classify maintenance reports using severity, location, and frequency, then rank reports by predefined rules.

Objective

Use tuples, conditions, loops, and list-based ranking.

Assumptions

• Severity is 1 (low) to 5 (critical).

• Safety-critical locations receive a priority bonus.

• Reports occurring at least 3 times receive a frequency bonus.

Algorithm

1. Read incident tuples.

2. Calculate priority score from severity, location, and frequency.

3. Print each score and repeatedly select the highest unranked report.

4. Summarize high-priority incidents.

Flowchart

Start → Read reports → Calculate priority scores → Rank highest first → Summarize urgent cases → End

Pseudocode

```
START
SET incident tuples
FOR each incident
  SCORE = severity * 2
  ADD 3 if safety-critical location
  ADD 2 if frequency >= 3
STORE scores
SELECT highest unranked score repeatedly
DISPLAY ranked report list
STOP
```

Python Source Code

```
reports = [
    ("Broken stair rail", "Stairwell", 4, 2),
    ("Water leak", "Laboratory", 3, 4),
    ("Light out", "Library", 2, 1),
    ("Exposed wire", "Electrical room", 5, 1)
]
safety_locations = ["Stairwell", "Laboratory", "Electrical room"]
scored = []

for title, location, severity, frequency in reports:
    score = severity * 2
    if location in safety_locations:
        score += 3
    if frequency >= 3:
        score += 2
    scored.append((score, title, location, severity, frequency))

ranked = []
while len(ranked) < len(scored):
    best = None
    for item in scored:
        if item not in ranked and (best is None or item[0] > best[0]):
            best = item
    ranked.append(best)

print("Priority ranking:")
for position, item in enumerate(ranked, start=1):
    score, title, location, severity, frequency = item
    print(position, title, "|", location, "| score:", score)
print("Urgent incidents:", sum(1 for item in ranked if item[0] >= 10))
```

Test Cases and Sample Output

```
Priority ranking:
1 Exposed wire | Electrical room | score: 13
2 Broken stair rail | Stairwell | score: 11
3 Water leak | Laboratory | score: 11
4 Light out | Library | score: 4
Urgent incidents: 3
```

Conclusion

This program demonstrates use tuples, conditions, loops, and list-based ranking. The sample data can be changed to test additional cases.

98. University Academic Progression Audit

Problem Statement

Combine semester marks, pass thresholds, attendance, and prerequisites to determine student progression eligibility.

Objective

Apply nested lists, tuples, conditions, and loops to academic rule checking.

Assumptions

• A subject is passed with marks at least 40.

• Minimum attendance is 75%.

• All listed prerequisites must be completed.

Algorithm

1. Read subjects, marks, attendance, and prerequisite status.

2. Check each subject's pass condition.

3. Check attendance and prerequisites.

4. Print eligibility and outstanding requirements.

Flowchart

Start → Read academic data → Check marks → Check attendance → Check prerequisites → Decide eligibility → End

Pseudocode

```
START
SET subject tuples and attendance
SET prerequisite status
FOR each subject
  IF marks < 40, record failed subject
IF attendance < 75, record attendance issue
IF prerequisite incomplete, record prerequisite issue
IF no issues, eligible; ELSE not eligible
DISPLAY audit result
STOP
```

Python Source Code

```
subjects = [
    ("Programming", 76),
    ("Mathematics", 35),
    ("AI Basics", 62)
]
attendance = 82
prerequisites = [("Programming Lab", True), ("Math Foundation", False)]
issues = []

for subject, marks in subjects:
    if marks < 40:
        issues.append("Failed subject: " + subject)
if attendance < 75:
    issues.append("Attendance below 75%")
for prerequisite, completed in prerequisites:
    if not completed:
        issues.append("Incomplete prerequisite: " + prerequisite)

if len(issues) == 0:
    print("Progression status: ELIGIBLE")
else:
    print("Progression status: NOT ELIGIBLE")
    for issue in issues:
        print("-", issue)
```

Test Cases and Sample Output

```
Progression status: NOT ELIGIBLE
- Failed subject: Mathematics
- Incomplete prerequisite: Math Foundation
```

Conclusion

This program demonstrates apply nested lists, tuples, conditions, and loops to academic rule checking. The sample data can be changed to test additional cases.

99. Integrated E-Commerce Order Processing and Reconciliation

Problem Statement

Process product orders, discounts, dispatches, returns, stock changes, and revenue reconciliation.

Objective

Use lists, tuples, loops, and arithmetic to maintain inventory and calculate net sales.

Assumptions

• Orders are (product, quantity, unit price, discount percent).

• Only dispatched quantities count as sales.

• Returned items are added back to stock and refunded at the discounted unit price.

Algorithm

1. Initialize stock and order records.

2. Calculate discounted order values.

3. Subtract dispatched quantities from stock.

4. Add returns back to stock and subtract refunds from revenue.

5. Compare expected revenue with collected revenue.

Flowchart

Start → Load stock/orders → Apply discount → Update dispatch stock → Process returns → Reconcile revenue → Report inventory → End

Pseudocode

```
START
SET inventory and orders
FOR each order
  CALCULATE discounted unit price
  SUBTRACT dispatched quantity from stock
  ADD dispatched sales to revenue
FOR each return
  ADD returned quantity to stock
  SUBTRACT refund from revenue
COMPARE collected and expected revenue
DISPLAY final stock and discrepancy
STOP
```

Python Source Code

```
stock = [["Mouse", 30], ["Keyboard", 20]]
orders = [
    ("Mouse", 5, 500, 10, 5),
    ("Keyboard", 3, 1000, 5, 3)
]  # product, ordered qty, unit price, discount %, dispatched qty
returns = [("Mouse", 1, 450)]
expected_revenue = 0

for product, ordered, price, discount, dispatched in orders:
    unit_net = price * (100 - discount) / 100
    expected_revenue += unit_net * dispatched
    for item in stock:
        if item[0] == product:
            item[1] -= dispatched

for product, quantity, refund_per_unit in returns:
    expected_revenue -= quantity * refund_per_unit
    for item in stock:
        if item[0] == product:
            item[1] += quantity

collected_revenue = 5900
print("Expected net revenue:", expected_revenue)
print("Collected revenue:", collected_revenue)
print("Discrepancy:", collected_revenue - expected_revenue)
print("Final inventory:", stock)
```

Test Cases and Sample Output

```
Expected net revenue: 4650.0
Collected revenue: 5900
Discrepancy: 1250.0
Final inventory: [['Mouse', 26], ['Keyboard', 17]]
```

Conclusion

This program demonstrates use lists, tuples, loops, and arithmetic to maintain inventory and calculate net sales. The sample data can be changed to test additional cases.

100. Comprehensive Smart Campus Management Simulation

Problem Statement

Build a menu-driven campus simulation for student records, attendance, marks, event registration, resource allocation, and summary reporting.

Objective

Integrate strings, lists, nested lists, tuples, tuple unpacking, loops, and conditional statements.

Assumptions

• The sample program starts with two student records.

• Menu choices are entered as numbers 1–6; choice 6 exits.

• Attendance is stored as attended and total classes; marks are out of 100.

Algorithm

1. Initialize student records, event registrations, and resources.

2. Display the menu repeatedly.

3. Perform the selected operation: list students, update attendance, update marks, register for event, allocate resource, or summary.

4. Exit when the user selects 6.

Flowchart

Start → Initialize records → Display menu → Read choice → Execute selected action → Return to menu? → End

Pseudocode

```
START
INITIALIZE students, events, and resources
REPEAT
  DISPLAY menu
  READ choice
  IF choice = 1, display students
  ELSE IF choice = 2, update attendance
  ELSE IF choice = 3, update marks
  ELSE IF choice = 4, register event
  ELSE IF choice = 5, allocate resource
  ELSE IF choice = 6, display summary and exit
  ELSE display invalid choice
UNTIL choice = 6
STOP
```

Python Source Code

```
students = [
    ["S101", "Asha", 18, 20, 78, []],
    ["S102", "Ravi", 15, 20, 66, []]
]
events = ["Tech Fest", "AI Workshop"]
resources = [["Projector", 3], ["Laptop", 10]]

while True:
    print("\n1. View students  2. Update attendance  3. Update marks")
    print("4. Register event  5. Allocate resource  6. Summary and exit")
    choice = input("Enter choice: ")

    if choice == "1":
        for sid, name, attended, total, marks, registered in students:
            print(sid, name, "attendance:", attended, "/", total, "marks:", marks)
    elif choice == "2":
        sid = input("Student ID: ")
        for student in students:
            if student[0] == sid:
                student[2] = int(input("Classes attended: "))
                student[3] = int(input("Total classes: "))
                print("Attendance updated.")
    elif choice == "3":
        sid = input("Student ID: ")
        for student in students:
```

```
            if student[0] == sid:
                student[4] = int(input("New marks: "))
                print("Marks updated.")
    elif choice == "4":
        sid = input("Student ID: ")
        print("Events:", events)
        event = input("Event name: ")
        for student in students:
            if student[0] == sid and event in events:
                student[5].append(event)
                print("Registration saved.")
    elif choice == "5":
        resource = input("Resource name: ")
        quantity = int(input("Quantity needed: "))
        for item in resources:
            if item[0] == resource:
                if quantity <= item[1]:
                    item[1] -= quantity
                    print("Allocated.")
                else:
                    print("Insufficient stock.")
    elif choice == "6":
        print("Campus summary")
        print("Students:", len(students))
        print("Event registrations:", sum(len(s[5]) for s in students))
        print("Resource balance:", resources)
        break
    else:
        print("Invalid choice. Try again.")
```

Test Cases and Sample Output

```
Test 1: Enter 1 to view students, then 6 to exit.
Expected: Student IDs S101 and S102 with their attendance and marks are displayed; summary reports 2 students.
```

Conclusion

This program demonstrates integrate strings, lists, nested lists, tuples, tuple unpacking, loops, and conditional statements. The sample data can be changed to test additional cases.